Dairy Management System – Project Requirements Document

Oct 27, 2026 · @Gaurang

1. Overview

A web-based Dairy Management System that lets a buffalo dairy record daily milk production per buffalo, track milk stock, sell to customers, generate invoices, log expenses and see daily and monthly profit, with two roles: Admin and Operator.

Goals

- Register buffaloes and keep an accurate live count.
- Record morning and evening milk per buffalo; totals are calculated automatically.
- Track stock: produced, sold, wasted and remaining.
- Manage customers, daily sales and printable invoices.
- Track expenses and report daily and monthly profit.
- Restrict sensitive data and actions by role.

Out of scope (future)

Payment gateway, SMS/WhatsApp notifications, feed and health tracking per buffalo, multi-farm support, mobile app, milk fat/SNF-based pricing.

2. Assumptions, constraints and stakeholders

The design serves one dairy farm run by a small team. These assumptions keep the scope realistic for an assignment.

Assumptions

- One farm, one currency (INR), one time zone (Asia/Kolkata).
- Milk is measured in litres, to two decimal places, and sold at a rate per litre.
- Each buffalo is milked twice a day (morning and evening); extra sessions are out of scope.
- Sales are recorded per customer per day; a customer may buy several times a day.
- Milk produced on a day is stocked for that day and carried forward as the next day's opening stock.
- Users have a stable internet connection while entering data.

Constraints

- Built as a basic working application within assignment time, so reports and PDFs stay simple.
- Only two roles; no per-user custom permissions.
- No payment gateway, no hardware (milk meters) integration, and no outgoing email: email is only the login identity in this version.

Stakeholders

Stakeholder	Interest	Uses the system as
Farm owner	Profit, expenses, control of users	Admin
Farm supervisor or staff	Quick daily entry of milk, sales and customers	Operator
Customers	Accurate invoices and balances	Receive invoices (no login)
Evaluator	Database design, calculations, inventory logic, code quality	Reviews the project

3. Tech stack and system design

A three-tier web app: React talks to a REST API, which reads and writes PostgreSQL. The API can be built in two ways: Option A with FastAPI (Python) or Option B with the PERN stack (PostgreSQL, Express.js, React, Node.js). Both expose the same endpoints, so the frontend and database stay the same.

Option A: React + FastAPI + PostgreSQL

Layer	Choice	Why
Frontend	React (Vite) + Tailwind CSS + Recharts	Fast to build, you already know it; charts for reports
Backend	FastAPI (Python) + Pydantic	Auto API docs, validation, clean calculation logic
ORM and migrations	SQLAlchemy + Alembic	Models mirror the schema; versioned changes
Database	PostgreSQL (SQLite for quick local dev)	Constraints, transactions, aggregate queries

Layer	Choice	Why
Auth	JWT access tokens, bcrypt password hashing	Stateless; role stored in the token
Invoices	ReportLab or WeasyPrint	Server-side PDF generation
Dev tooling	Docker Compose, pytest, Swagger UI	One-command setup; testable calculations

Option B: PERN (PostgreSQL, Express.js, React, Node.js)

Choose this if you prefer JavaScript end to end. The database, frontend, endpoints and architecture are the same as Option A; only the backend changes. PostgreSQL remains the main database, with SQLite for quick local development.

Layer	Choice	Why
Frontend	React (Vite) + Tailwind CSS + Recharts	Unchanged from Option A
Backend	Node.js (LTS) + Express.js	One language across frontend and backend
ORM and migrations	Prisma + Prisma Migrate (Sequelize also works)	Typed models mirror the schema; versioned changes
Database	PostgreSQL (SQLite for quick local dev)	Unchanged from Option A
Validation	Zod (or express-validator)	Request validation, the role Pydantic plays in Option A
Auth	jsonwebtoken + bcrypt, role middleware	Same JWT and role flow; login by email
Invoices	PDFKit	Server-side PDF generation
Dev tooling	Docker Compose, Jest + Supertest, swagger-ui-express	One-command setup; testable calculations

Option A or Option B

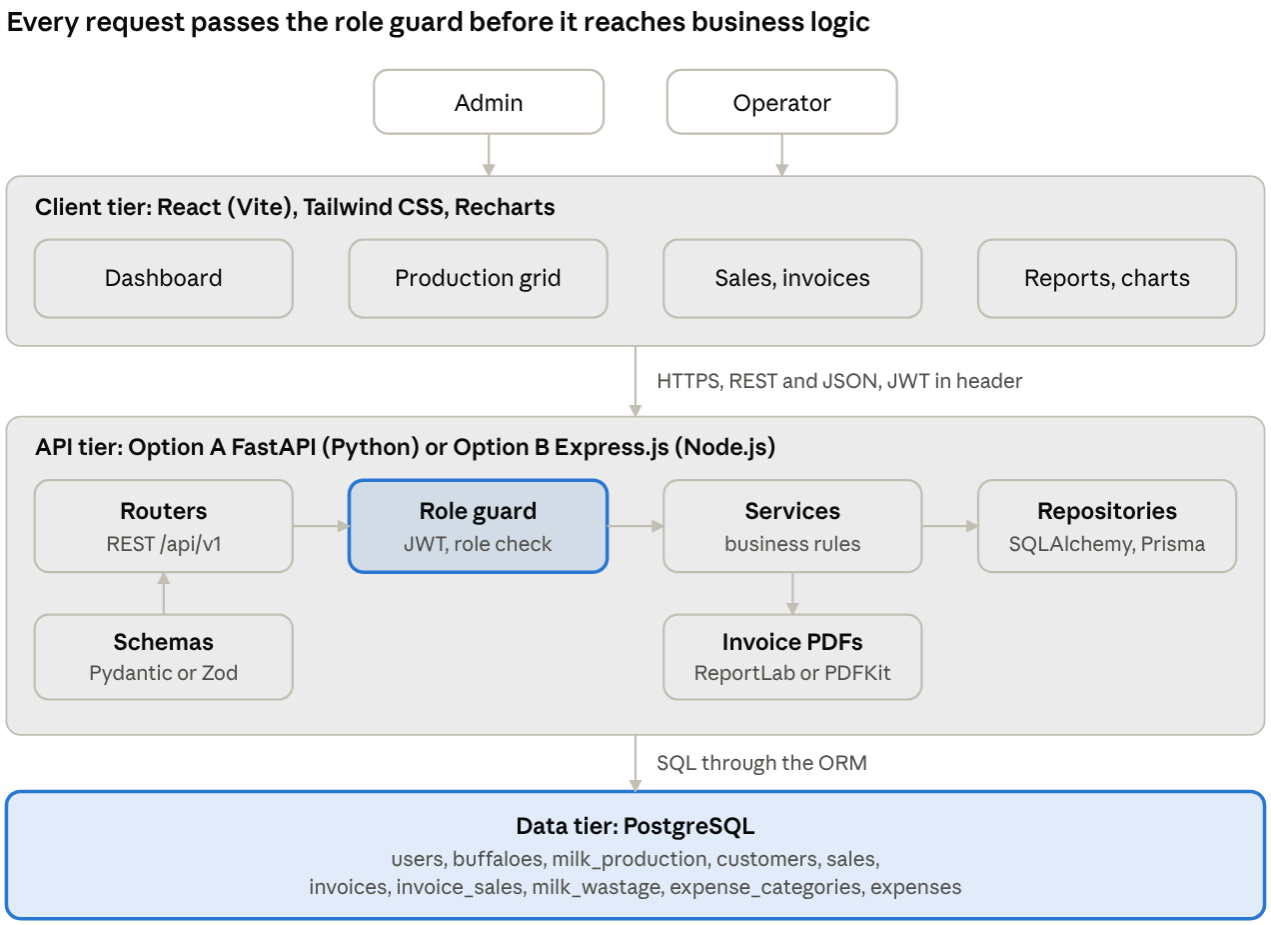
Question	Option A: FastAPI	Option B: Express.js
Language	Python	JavaScript or TypeScript

Question	Option A: FastAPI	Option B: Express.js
Pick it if	You like Python and want automatic API docs	You want one language everywhere
Validation and API docs	Built in (Pydantic, Swagger UI)	Added with Zod and swagger-ui-express
Money and litres	Numeric columns, Python Decimal	Prisma Decimal; never JavaScript floats
Extra setup	Less	Slightly more, same result

Both options meet every requirement in this document. Pick one before you start and keep to it.

Request flow: login with email and password returns a JWT with the user's role; every API call sends it; a FastAPI dependency (or Express middleware in Option B) checks the role before the endpoint runs.

System design



system design · 3 tiers, 4 API layers

The browser never touches the database. Routers receive and validate requests, the role guard rejects anyone without permission, services apply the stock, sales and profit rules, and repositories run the SQL.

4. Users, roles and permissions

Admin has full access. Operator handles day-to-day operations but cannot see users, expenses or profit data, and cannot delete records.

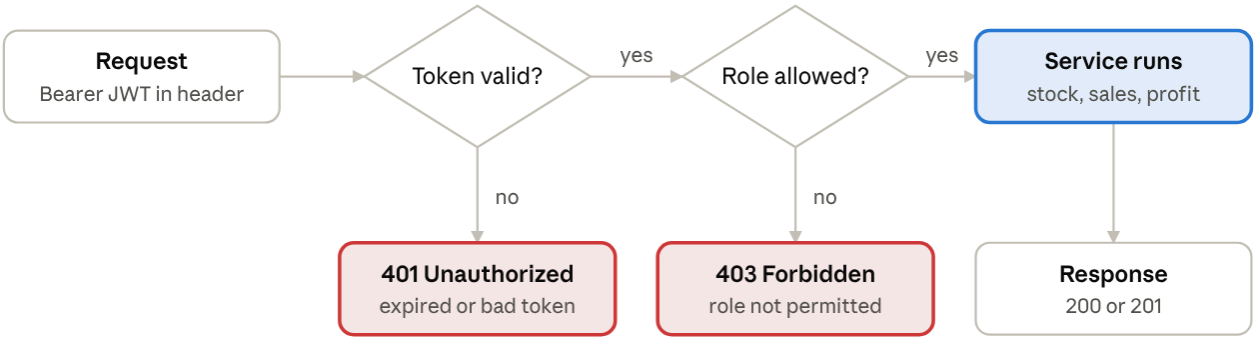
Module	Admin	Operator
User management (create, deactivate, reset password)	Full	No access
Buffaloes	Full	Add, view, edit
Milk production	Full	Add, view, edit own-day entries
Milk stock (sold, wasted, remaining)	Full	View, record wastage
Customers	Full	Add, view, edit
Sales and invoices	Full	Create, view, print
Expenses	Full	No access
Profit and financial reports	Full	No access
Production and stock reports	Full	View
Deleting any record	Yes (soft delete)	No

Rules

- Users log in with email and password; the email is unique and is the login identity, while username is only a display name. The role is fixed per user and set only by Admin.
- The first Admin is created by a seed script; there is no public sign-up.
- Permissions are enforced on the backend for every endpoint; the frontend only hides menus.
- Inactive users cannot log in. Passwords are bcrypt-hashed.
- Assumption: "essential data" for Operator means users, expenses and profit figures. Adjust the matrix if you want a different split.

Authorization flow

A request reaches business logic only after the token and role checks pass



Admin passes every guard; Operator gets 403 on users, expenses, profit and delete.

authorization flow · 2 checks, 2 error exits

The token check answers who the caller is; the role check answers what they may do. Only a request that passes both reaches a service.

5. Functional requirements

FR-1 Authentication and users (Admin)

- Log in with email and password to receive a JWT; log out; show the logged-in user and role.
- Admin creates, edits, deactivates users and assigns Admin or Operator; every user needs a unique, valid email.

FR-2 Buffalo management

- Register a buffalo: tag number (unique), name, breed, date of birth, purchase date, status (Active, Sold, Dry, Dead).
- List with search and status filter; edit details.
- Dashboard shows the count of active buffaloes and a count per status.

FR-3 Daily milk production

- Enter litres per buffalo per date for the Morning or Evening session; one entry per buffalo, date and session.
- A bulk entry grid lists all active buffaloes for a chosen date so the operator fills both sessions quickly.
- Auto-calculate each buffalo's daily total and the herd's morning total, evening total and grand total.
- Only active buffaloes can be entered; litres must be zero or more.

FR-4 Milk stock

- Daily stock record: opening stock, produced, sold, wasted, closing stock.
- Record wastage with litres and a reason (spoiled, spilled, returned).
- Warn and block when a sale would exceed available stock.

FR-5 Customers

- Add and edit customers: name, phone, address, price per litre (default or custom), status.
- View a customer's sales history and outstanding balance.

FR-6 Sales and invoices

- Record a daily sale: customer, date, quantity in litres, rate; amount is calculated.
- Generate an invoice per sale or a monthly invoice per customer, with a unique invoice number.
- Download or print the invoice as PDF; mark as Paid, Partial or Unpaid.

FR-7 Expenses (Admin)

- Record an expense: date, category (feed, labour, medicine, electricity, transport, other), amount, note.
- Filter by date range and category.

FR-8 Reports and dashboard

- Dashboard: today's production, stock, sales and active buffalo count.
- Buffalo-wise production report by date range.
- Daily and monthly profit report (Admin): revenue, expenses, profit.
- Export reports to CSV or PDF.

6. Use cases and acceptance criteria

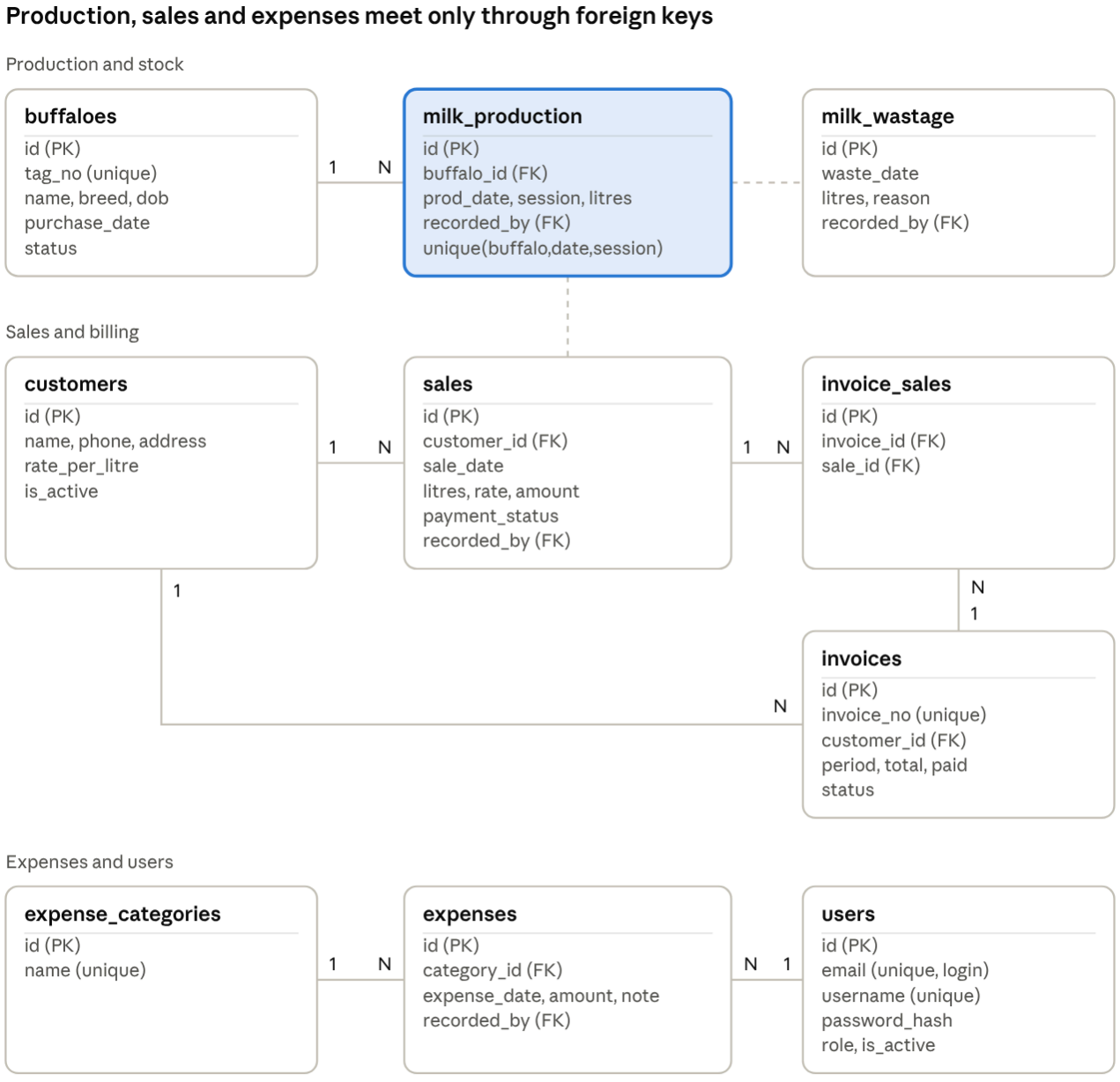
Each use case is done when its acceptance criteria pass.

ID	Use case	Role	Acceptance criteria
UC-1	Log in	Admin, Operator	Valid email and password return a token and open the dashboard; a wrong email or password shows one generic error; a deactivated user cannot log in

ID	Use case	Role	Acceptance criteria
UC-2	Register a buffalo	Admin, Operator	Tag number must be unique; the new buffalo appears in the list and raises the active count by 1
UC-3	Record milk production	Admin, Operator	One row per buffalo, date and session; saving a duplicate asks to update; morning, evening and day totals update immediately
UC-4	View buffalo-wise production	Admin, Operator	For a date range, each buffalo shows morning, evening and total litres; the sum of rows equals the herd total
UC-5	Record wastage	Admin, Operator	Litres and reason are required; wastage above available stock is rejected; closing stock drops by the wasted amount
UC-6	Add a customer	Admin, Operator	Name and rate are required; the customer is available in the sales form
UC-7	Record a sale	Admin, Operator	Amount equals litres x rate; a sale above available stock is rejected; stock sold increases by the sale litres
UC-8	Generate an invoice	Admin, Operator	Invoice number is unique; total equals the sum of the selected sales; the PDF downloads and shows customer, period and balance
UC-9	Record an expense	Admin	Category, date and amount are required; the expense appears in the daily and monthly totals
UC-10	View profit report	Admin	Profit equals revenue minus expenses for the chosen day or month; Operator gets a 403 response
UC-11	Manage users	Admin	Admin can create, deactivate and change a user's role; a duplicate email is rejected with 409; Operator gets a 403 response on every user endpoint
UC-12	Delete a record	Admin	The record is soft-deleted, hidden from lists and excluded from totals; Operator has no delete option

7. Database design

Ten tables in PostgreSQL. Every table has `id` (PK), `created_at`, `updated_at`; deletable tables also have `is_deleted`.



Solid line: foreign key, 1 to many. Dashed line: both feed the derived daily stock.
milk_production, sales and milk_wastage also store recorded_by, a foreign key to users.

ER diagram · 10 tables

Read it in three bands: production and stock, sales and billing, expenses and users. Every row in a transaction table points to exactly one parent row.

Table	Key columns	Notes
users	full_name, username (unique), email (unique, login), password_hash, role (admin/operator), is_active	Email is the login identity; roles enforced in API
buffaloes	tag_no (unique), name, breed, dob, purchase_date, status	Count = active rows
milk_production	buffalo_id (FK), prod_date, session (morning/evening), litres, recorded_by (FK users)	Unique (buffalo_id, prod_date, session); litres >= 0
customers	name, phone, address, rate_per_litre, is_active	
sales	customer_id (FK), sale_date, litres, rate, amount, payment_status, recorded_by	amount = litres x rate
invoices	invoice_no (unique), customer_id (FK), period_start, period_end, total_amount, paid_amount, status	Links to sales via invoice_sales
invoice_sales	invoice_id (FK), sale_id (FK)	Many sales per invoice
milk_wastage	waste_date, litres, reason, recorded_by	Reduces stock
expense_categories	name (unique)	Feed, labour, medicine, etc.
expenses	category_id (FK), expense_date, amount, note, recorded_by	Admin only

Relationships

- One buffalo has many milk_production rows (one per date and session).
- One customer has many sales and many invoices; one invoice groups many sales.
- One expense category has many expenses.
- Users record production, sales, wastage and expenses.

Stock is derived, not typed in. Daily stock is computed from production, sales and wastage; an optional `daily_stock` snapshot table can cache closing stock for speed.

Indexes: (prod_date), (buffalo_id, prod_date), (sale_date), (customer_id, sale_date), (expense_date); unique indexes on users.email, users.username, buffaloes.tag_no and invoices.invoice_no.

8. Calculations

All totals are computed from stored entries with SQL aggregates, so they can never drift out of sync.

Buffalo-wise daily production

```
Total_{b,d} = Morning_{b,d} + Evening_{b,d}
```

```
SELECT buffalo_id, prod_date,
       SUM(litres) FILTER (WHERE session = 'morning') AS morning,
       SUM(litres) FILTER (WHERE session = 'evening') AS evening,
       SUM(litres) AS total
FROM milk_production
WHERE prod_date BETWEEN :from AND :to
GROUP BY buffalo_id, prod_date;
```

Herd totals for a day

```
Produced_d = \sum_{b} (Morning_{b,d} + Evening_{b,d})
```

Milk stock

```
Closing_d = Opening_d + Produced_d - Sold_d - Wasted_d
```

```
Opening_d = Closing_{d-1}
```

A sale or wastage entry is rejected if it would make closing stock negative.

Sales and invoice

```
Amount = Litres \times Rate
```

```
Invoice\ total = \sum Amount_{sales\ in\ period}
```

```
Balance = Invoice\ total - Paid\ amount
```

Profit

```
Profit_{period} = Sales\ revenue_{period} - Expenses_{period}
```

Daily profit uses the day's sales and expenses; monthly profit sums every day in the month. Monthly expenses are not spread across days unless Admin chooses a daily-average view.

Average yield per buffalo

```
Avg_b = \frac{\sum Total_{b,d}}{Days\ recorded_b}
```

Worked example for one day

Buffalo	Morning (L)	Evening (L)	Day total (L)
B-01	6.5	5.5	12.0
B-02	5.5	5.0	10.5
B-03	4.5	4.5	9.0
Herd	16.5	15.0	31.5

With opening stock 20.0 L, sales of 40.0 L and wastage of 2.0 L, closing stock is 20.0 + 31.5 - 40.0 - 2.0 = 9.5 L. At a rate of 60 per litre, sales revenue is 2,400; with 900 of expenses, the day's profit is 1,500.

9. Low-level design

The backend uses four layers so each rule lives in exactly one place: routers, services, repositories and models.

Option A: FastAPI backend and React frontend structure

```
backend/
  app/
    main.py                # FastAPI app, router registration
    core/                  # config.py, security.py (JWT, bcrypt), deps.py
(current user, role guard)
    db/                    # session.py, base.py
    models/                # SQLAlchemy tables
    schemas/               # Pydantic request and response models
    repositories/          # database queries only
    services/              # business rules: stock, sales, profit, invoices
    routers/               # HTTP endpoints, one file per module
    utils/pdf.py           # invoice PDF builder
  alembic/                 # migrations
  tests/                   # pytest
frontend/
```

```
  src/
    api/                    # axios client, one file per module
    context/AuthContext.jsx # token, user, role
    routes/                 # protected routes by role
    pages/                  # Dashboard, Buffaloes, Production, Sales,
Reports, Users
    components/             # tables, forms, charts
```

Option B: Express.js backend structure

```
backend/
  src/
    app.js                  # Express app, router registration
    server.js               # starts the HTTP server
    config/                 # env.js, db.js
    middleware/             # auth.js (verify JWT), requireRoles.js,
errorHandler.js
    validators/             # Zod schemas
    repositories/           # database queries only (Prisma client)
    services/               # business rules: stock, sales, profit, invoices
    routes/                 # Express routers, one file per module
    utils/pdf.js            # PDFKit invoice builder
  prisma/                   # schema.prisma (models) and migrations
  tests/                    # Jest + Supertest
```

The React frontend and the four-layer rules below are identical in both options.

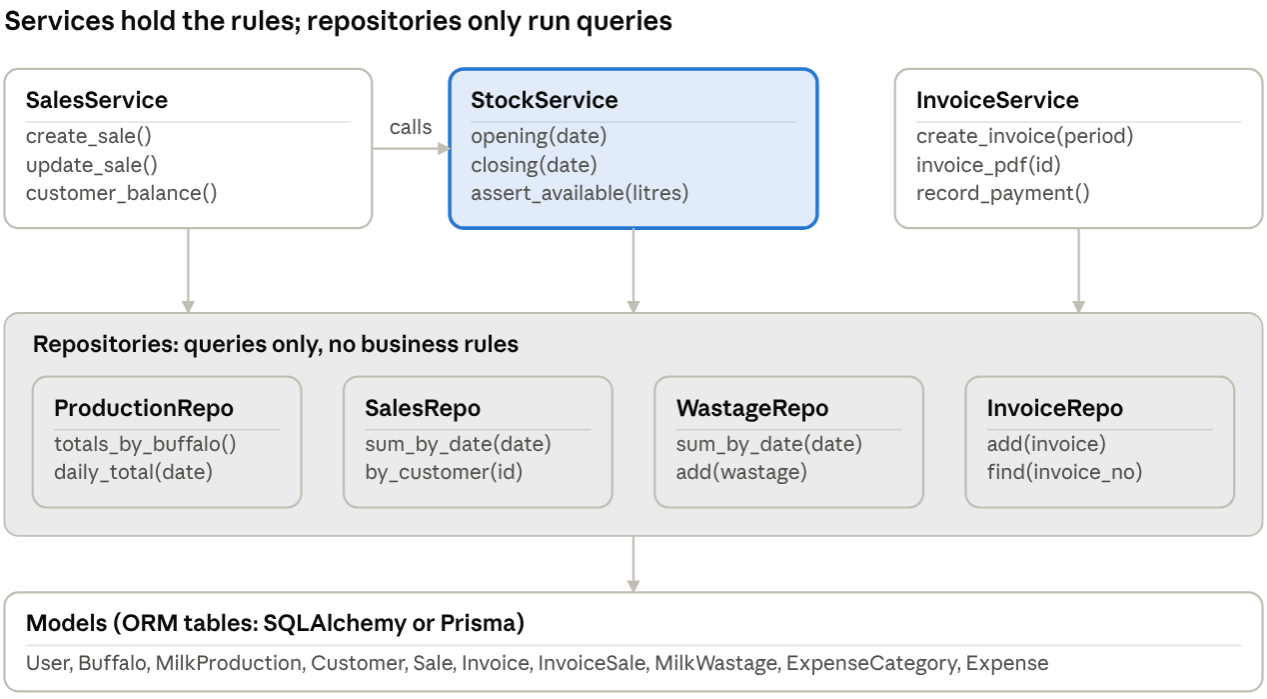
Layer responsibilities

Layer	Responsibility	Must not
Router	Parse request, call the role guard, call one service, return the response	Contain business rules or SQL
Schema	Validate input and shape output	Touch the database
Service	Stock checks, amount calculation, invoice totals, profit; owns the transaction	Build HTTP responses
Repository	Queries and aggregates through SQLAlchemy (Option A) or Prisma (Option B)	Decide permissions or rules
Model	Table definition, constraints, relationships	Hold logic beyond simple properties

Design decisions

- Role guard is a FastAPI dependency: `Depends(require_roles('admin'))` on admin-only routes; Option B uses an Express middleware, requireRoles('admin'), the same way.
- Stock is calculated in `StockService`; `SalesService` and `WastageService` call it before saving.
- A sale and its stock check run in one database transaction, so two operators cannot oversell the same litres.
- Money uses `Numeric(12,2)` and litres use `Numeric(10,2)`; never floating point (Prisma Decimal in Option B).
- Dates are stored as `DATE`; timestamps in UTC and shown in Asia/Kolkata.

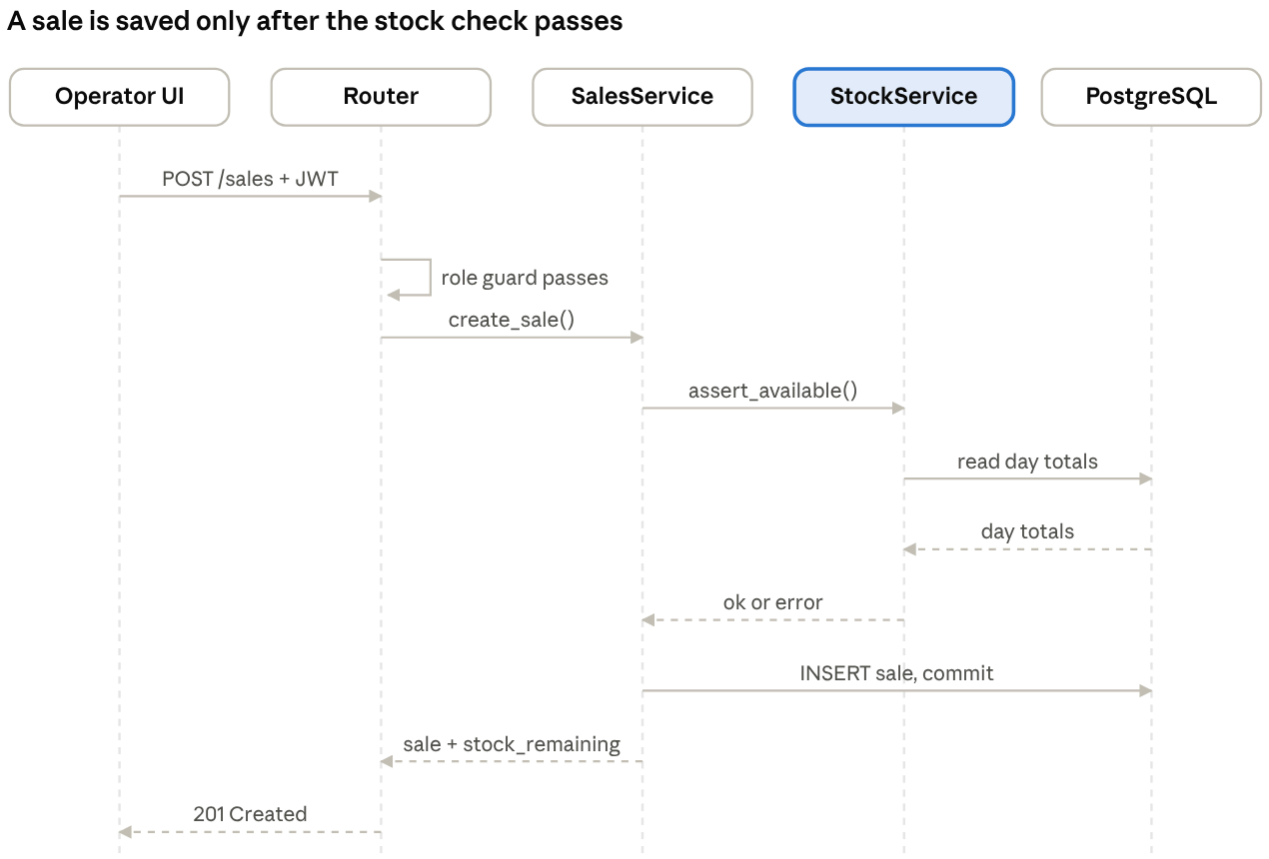
Class diagram



class diagram · 3 services, 4 repositories

SalesService calls StockService before it saves a sale. StockService reads production, sales and wastage totals; InvoiceService reads sales and writes invoices, and uses `utils/pdf.py` for the PDF. Repositories are shortened to Repo in the picture. Method names are shown in Python style; in Option B they use camelCase, for example createSale().

Sequence: recording a sale



If the stock check fails, nothing is saved and the API answers 400 with STOCK_INSUFFICIENT.

sequence diagram · record a sale, 5 participants

The stock check and the insert run in one database transaction, so a failed check leaves no partial sale. Recording milk production and wastage follow the same path, with their own service in place of SalesService.

10. API design

REST under `/api/v1`, JSON in and out, JWT required except login.

Module	Endpoints	Access
Auth	POST /auth/login (email, password), GET /auth/me	All
Users	GET, POST /users; PUT /users/{id}; PATCH /users/{id}/status	Admin
Buffaloes	GET, POST /buffaloes; GET, PUT /buffaloes/{id}; GET /buffaloes/count	Admin, Operator

Module	Endpoints	Access
Production	POST /production (single or bulk); GET /production?date=; GET /production/summary?from=&to=; GET /production/buffalo/{id}	Admin, Operator
Stock	GET /stock?date=; POST /wastage; GET /wastage	Admin, Operator
Customers	GET, POST /customers; GET, PUT /customers/{id}; GET /customers/{id}/balance	Admin, Operator
Sales	GET, POST /sales; GET, PUT /sales/{id}	Admin, Operator
Invoices	POST /invoices; GET /invoices; GET /invoices/{id}/pdf; PATCH /invoices/{id}/payment	Admin, Operator
Expenses	GET, POST /expenses; PUT /expenses/{id}; GET /expense-categories	Admin
Reports	GET /reports/profit?period=; GET /reports/production; GET /reports/dashboard	Profit: Admin; others: Admin, Operator
Delete	DELETE /{resource}/{id} (soft delete)	Admin

Conventions

- Success returns the resource or a list with `page`, `page_size` and `total`; creation returns 201.
- Errors use one shape: `{"error": {"code": "STOCK_INSUFFICIENT", "message": "Only 9.5 L available"}}`.
- Status codes: 400 validation or business-rule error, 401 missing or invalid token, 403 role not allowed, 404 not found, 409 duplicate.
- Dates use ISO format (`2026-10-27`); litres and money are decimal strings with two places.
- List endpoints accept `from`, `to`, `search` and `status` filters where they make sense.
- Interactive documentation is available at `/docs` (Swagger UI) in FastAPI; Option B serves the same page through swagger-ui-express.

Sample: record a sale

```
POST /api/v1/sales
{ "customer_id": 4, "sale_date": "2026-10-27", "litres": "12.00", "rate":
"60.00" }

201 Created
{ "id": 118, "amount": "720.00", "payment_status": "unpaid",
"stock_remaining": "9.50" }
```

11. Non-functional requirements, validations and edge cases

Non-functional

- Security: bcrypt hashes, JWT expiry (8 hours), role checks on every endpoint, HTTPS in deployment, no secrets in code, and one generic error for a wrong email or password.
- Performance: dashboard and list pages load in under 2 seconds for one year of data.
- Reliability: stock-changing writes run in database transactions; daily database backup.
- Usability: responsive layout so operators can enter data on a phone; bulk entry grid for production.
- Auditability: every record stores who created it and when; deletes are soft deletes.
- Maintainability: unit tests for stock, invoice and profit calculations.

Validations

- Litres and amounts must be zero or more; rate must be greater than zero.
- One production entry per buffalo, date and session; a duplicate updates the existing entry after confirmation.
- Production date cannot be in the future; Operator can edit entries only for the current day, Admin for any day.
- Tag number, invoice number and user email are unique; email must be a valid address and is stored in lower case.

Edge cases

- A sold or dead buffalo keeps its history but is hidden from new entry grids.
- Selling more than available stock is blocked with a clear message.
- Editing past production or sales recalculates the closing stock of every later day.
- Invoices for a period that already has an invoice for the same sales are rejected.
- A deactivated customer keeps old invoices but cannot get new sales.

12. Testing strategy

The riskiest parts are the calculations and the role checks, so they get automated tests first. The tools below are for Option A; Option B uses Jest and Supertest with the same cases.

Level	Tool	What it covers
Unit	pytest	Stock, amount, invoice total and profit functions with fixed inputs
API	pytest + FastAPI TestClient, test database	Every endpoint: success, validation error, 401 and 403 cases
Role	pytest	Operator gets 403 on users, expenses, profit and delete endpoints
Frontend	Manual checklist, optional Vitest	Forms, bulk entry grid, protected routes, PDF download

Key test cases

Case	Input	Expected
Buffalo daily total	Morning 6.5 L, evening 5.5 L	12.0 L
Herd total	Three buffaloes at 12.0, 10.5, 9.0 L	31.5 L
Stock closing	Opening 20, produced 31.5, sold 40, wasted 2	9.5 L
Oversell	Stock 9.5 L, sale of 10 L	Rejected with a stock error
Sale amount	12 L at 60 per litre	720.00
Profit	Revenue 5,400, expenses 3,900	1,500
Duplicate production	Same buffalo, date and session twice	Second save asks to update; no duplicate row
Operator on profit report	Operator token	403 Forbidden
Deactivated user	Valid email and password, inactive account	Login rejected

13. Deployment and configuration

The whole system starts with one `docker compose up` command, and the same containers can be deployed to a small VPS or a cloud platform.

Service	Image or build	Port	Notes
db	postgres:16	5432 (internal)	Named volume for data; daily `pg_dump` backup
backend	Python 3.12 + Uvicorn (Option A) or Node.js LTS + Express.js (Option B)	8000	Runs migrations on start (Alembic, or Prisma Migrate in Option B), then the API
frontend	Node build served by Nginx	80	Proxies `/api` to the backend

Environment variables (backend)

Variable	Purpose
DATABASE_URL	PostgreSQL connection string
JWT_SECRET	Secret used to sign tokens; long random value
JWT_EXPIRE_MINUTES	Token lifetime; 480 for 8 hours
ADMIN_EMAIL, ADMIN_PASSWORD	First Admin created by the seed script
CORS_ORIGINS	Allowed frontend origin
TZ	Asia/Kolkata

Run order: start db, run migrations, run the seed script (first Admin and expense categories), start backend, start frontend. Secrets stay in a `.env` file that is never committed.

14. Risks and mitigations

Risk	Impact	Mitigation
Wrong stock because past entries are edited	Closing stock of later days is wrong	Compute stock from transactions; recalculate later days on every edit

Risk	Impact	Mitigation
Two users sell the same litres at once	Stock goes negative	Run stock check and insert in one transaction with a row lock on the day's stock
Duplicate production entries	Inflated totals	Unique constraint on buffalo, date and session
Operator sees financial data	Privacy and trust issue	Role guard on every endpoint; tests for every 403 case
Stolen or weak token	Unauthorised access	Short expiry, strong secret, HTTPS, bcrypt hashes
Rounding errors in money	Invoices do not add up	Numeric types with two decimals, never floats
Scope too large for the deadline	Incomplete submission	Build in the order of section 15; keep PDFs and charts simple
Data loss	Records lost	Daily database backup; soft deletes

15. UI screens, development plan and future scope

Screens

Screen	Purpose	Roles
Login	Authenticate and route by role	All
Dashboard	Today's production, stock, sales, buffalo count	All (profit widgets Admin only)
Buffaloes	List, add, edit, status	Admin, Operator
Milk production	Bulk entry grid by date; buffalo-wise report	Admin, Operator
Stock and wastage	Daily stock table, record wastage	Admin, Operator
Customers	List, add, edit, balance	Admin, Operator
Sales and invoices	New sale, invoice list, PDF download	Admin, Operator

Screen	Purpose	Roles
Expenses	Add and filter expenses	Admin
Reports	Production report; profit report with charts	Profit: Admin
Users	Manage users and roles	Admin

Development plan

1. Set up repo, Docker Compose, PostgreSQL, the backend (FastAPI or Express.js) and React skeletons.
2. Create models and migrations; seed the first Admin (email and password from environment variables) and expense categories.
3. Build email and password login with JWT and role guards.
4. Build buffaloes and milk production with the calculation queries.
5. Build stock, wastage, customers, sales and invoice PDFs.
6. Build expenses, dashboard and profit reports.
7. Write calculation tests, polish validations, add sample data and a README with an ER diagram.

Future scope

Email verification and password reset by email, feed and health records per buffalo, fat/SNF-based pricing, SMS or WhatsApp invoice sharing, online payments, multi-farm support, mobile app.

16. Assignment mapping

Every assignment requirement and evaluation criterion points to a section of this document.

Assignment item	Where it is covered
1. Buffalo registration and count	FR-2 (section 5), `buffaloes` table (section 7), UC-2 (section 6)
2. Daily milk production per buffalo	FR-3 (section 5), `milk_production` table (section 7)
3. Automatic morning and evening totals	Calculations (section 8), SQL example, UC-3

Assignment item	Where it is covered
4. Stock: sold, wasted, remaining	FR-4 (section 5), stock formula (section 8), wastage table
5. Customers, sales, invoices	FR-5 and FR-6 (section 5), `customers`, `sales`, `invoices` tables
6. Expenses and profit reports	FR-7 and FR-8 (section 5), profit formula (section 8)
Task: design the database	Section 7 with the ER diagram
Task: explain buffalo-wise calculation	Section 8, buffalo-wise daily production
Task: basic working application	Sections 3, 9, 10 and the development plan in section 15
Evaluation: database design	Section 7: normalised tables, keys, constraints, indexes
Evaluation: calculations	Section 8 and the test cases in section 12
Evaluation: inventory management	Stock rules in sections 5, 8 and 11
Evaluation: coding skills	Layered design in section 9, tests in section 12
Evaluation: problem-solving	Edge cases in section 11, risks in section 14

17. Glossary

Term	Meaning
Session	Morning or evening milking
Production	Litres of milk collected from a buffalo in one session
Wastage	Milk lost to spoilage, spillage or return; reduces stock
Opening and closing stock	Litres available at the start and end of a day
Rate	Selling price per litre for a customer
Invoice	Bill listing a customer's sales for a period, with total and balance

Term	Meaning
Soft delete	Marking a record as deleted without removing the row, so history and totals stay consistent
JWT	JSON Web Token, a signed token that proves who the user is and carries the role
RBAC	Role-based access control: permissions depend on the user's role
ER diagram	Entity-relationship diagram showing tables and how they connect
HLD, LLD	High-level design (system parts) and low-level design (classes and flows)

PERN: PostgreSQL, Express.js, React and Node.js. Option B of this document uses it in place of the FastAPI backend, with the same database, frontend and endpoints.